E9: 309 Advanced Deep Learning

4-11-2020

http://leap.ee.iisc.ac.in/sriram/teaching/ADL2020/

Housekeeping

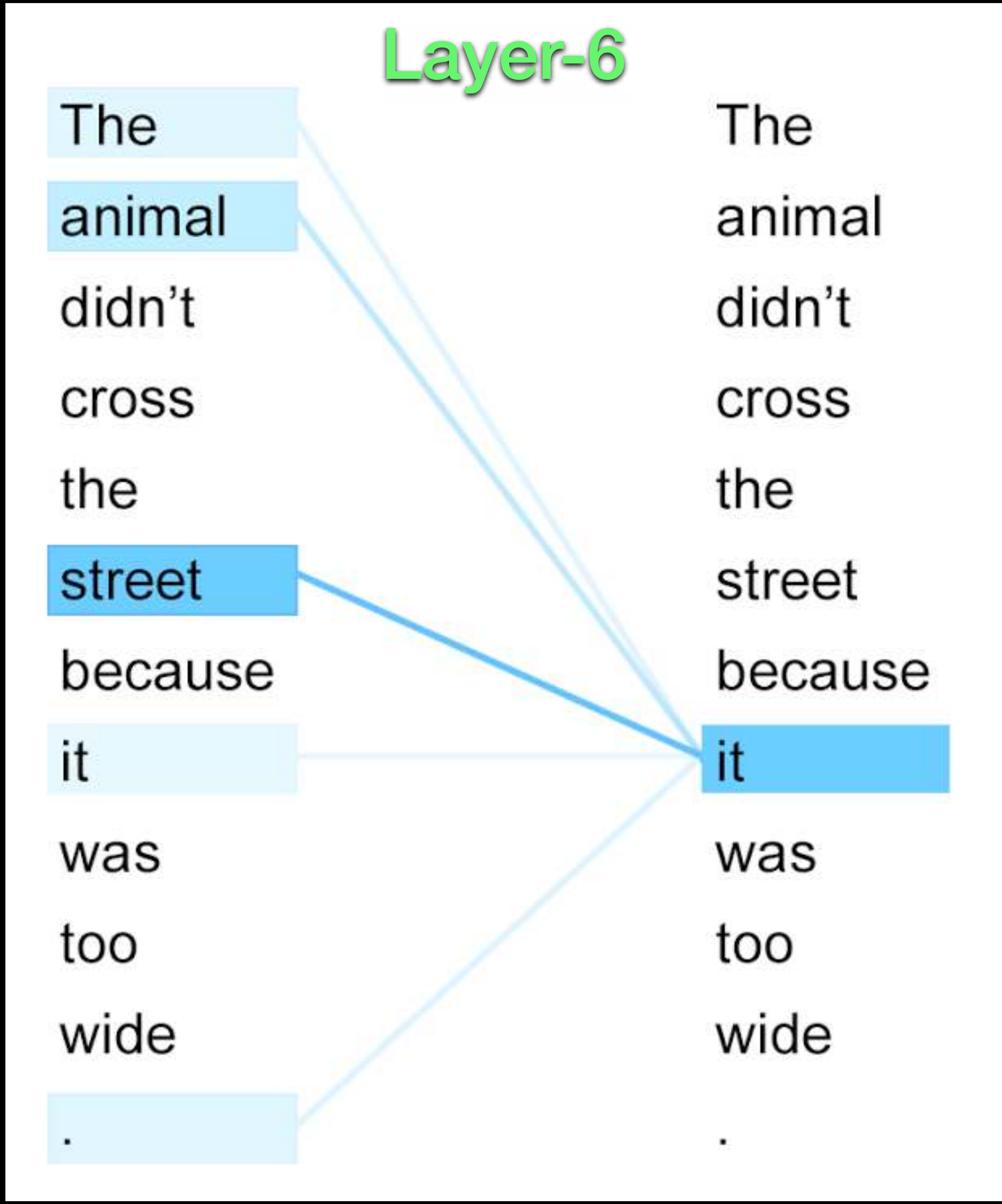
- 1st mini-project
  - ✓ Deadlines
    - ★ Abstract submission deadline (Nov 2nd, Monday)
      - ★ Using the google form given in the webpage
    - ★ Solo projects or 2-member projects
      - ★ Indicate roles of each member in 2-member project
      - ★ 200 page abstract of the work. If modifications are needed, we will review and let you know in 2-3 days.

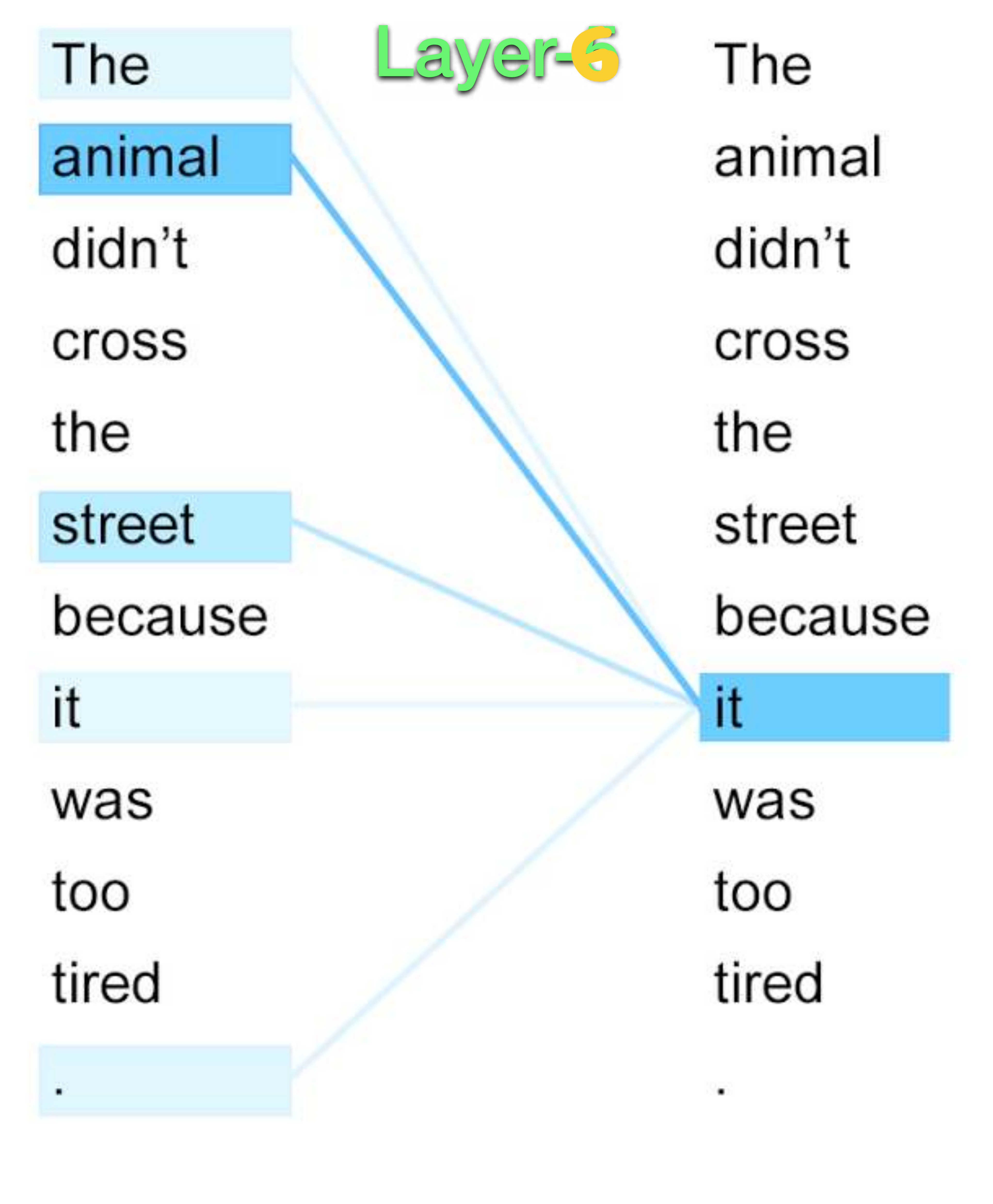
Recap of previous class

State of affairs

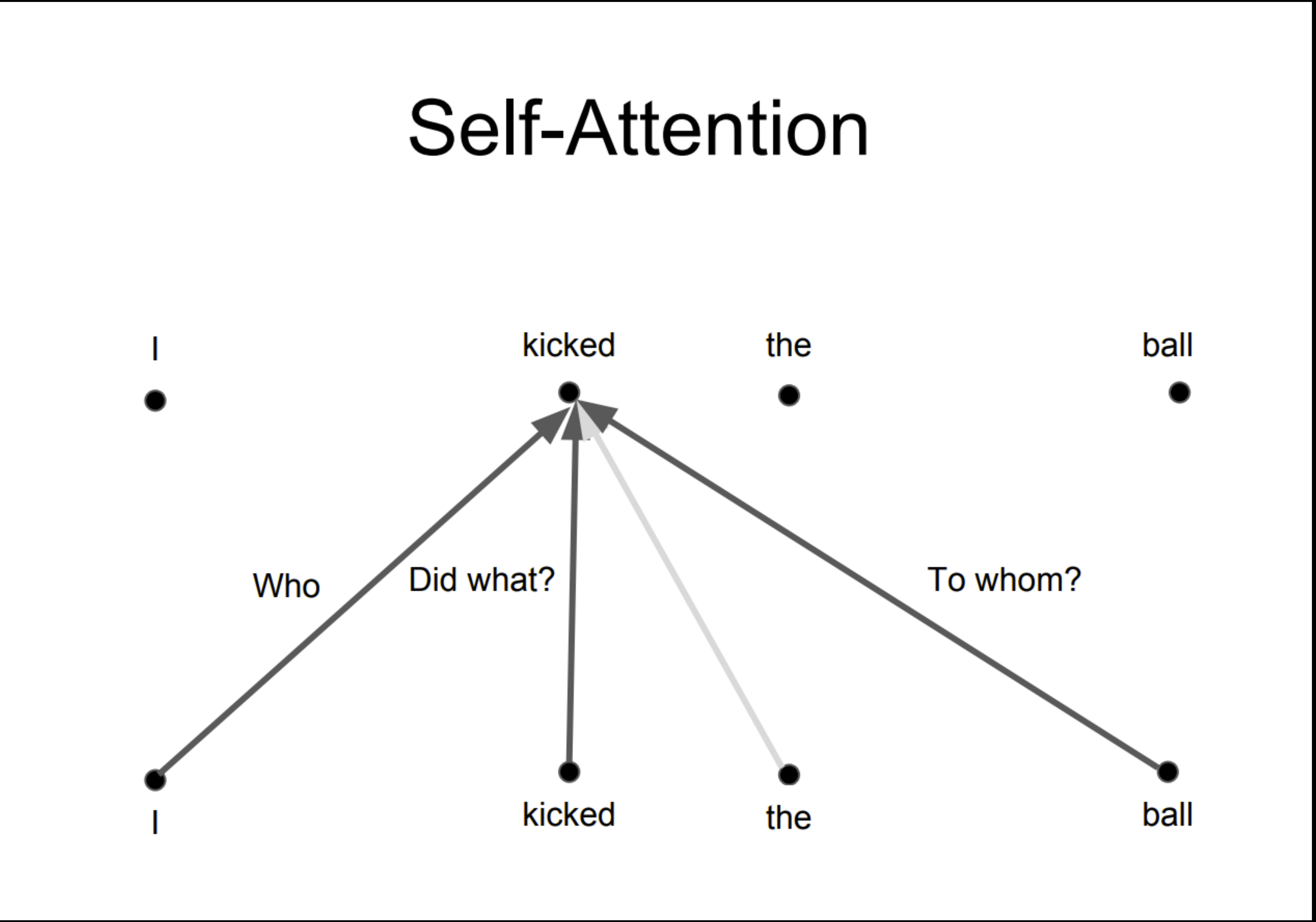
- Transformer models
  - ✓ Encoder-decoder model
    - ★ Repeated architecture in encoder and decoder stages
    - ★ Multi-head self-attention
    - ★ Position wise feed-forward.
    - ★ Skip connection with layer norm.

Self-attention - need for depth



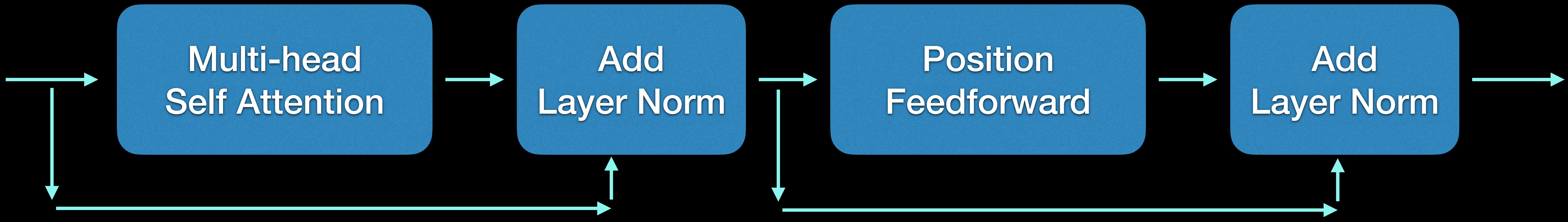


Need for multi-head attention



Single layer of encoder (typical implementation)

- Single encoder layer has typically self-attention skip connection, layer norm and feedforward layer



Positional encoding

- No recurrence or position awareness yet in the model

Binary format -
position can encode the rate of
change of bits across time

In floating format - one can use
sines and cosines

0 :	0	0	0	0	8 :	1	0	0	0
1 :	0	0	0	1	9 :	1	0	0	1
2 :	0	0	1	0	10 :	1	0	1	0
3 :	0	0	1	1	11 :	1	0	1	1
4 :	0	1	0	0	12 :	1	1	0	0
5 :	0	1	0	1	13 :	1	1	0	1
6 :	0	1	1	0	14 :	1	1	1	0
7 :	0	1	1	1	15 :	1	1	1	1

Positional encoding

- An example used in the first paper

$$\mathbf{p}(t) \in \mathcal{R}^D$$

$i = 1 \dots D$

$$p_i(t) = \begin{cases} sin(\omega_k t), & \text{if } i = 2k \\ cos(\omega_k t), & \text{if } i = 2k+1 \end{cases}$$

$$k \in \{1 \dots \frac{D}{2}\}$$

$$\omega_k = \frac{1}{10000^{\frac{2k}{D}}}$$

model

$t = 1 \dots T$

$$\mathbf{x}(t) = \mathbf{x}(t) + \mathbf{p}(t)$$

$x(1) \; x(2) \; \dots \; x(T)$

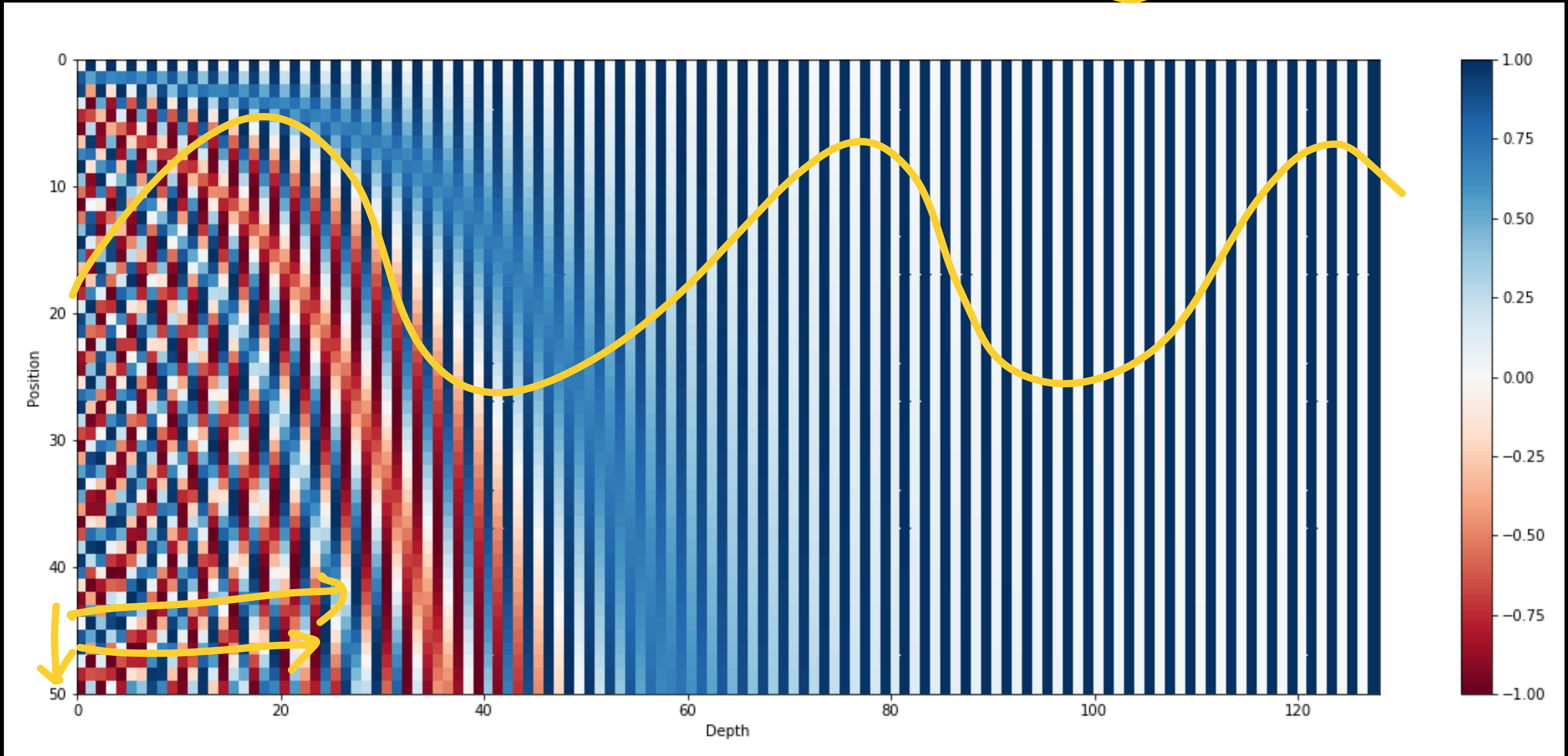
$X = \{x(1) \dots x(T)\} \in \mathbb{R}^{T \times D}$

Positional encoding

T=20

T=100

- An example used in the first paper $\mathbf{p}(t) \in \mathcal{R}^D$ [T=50, D=128]



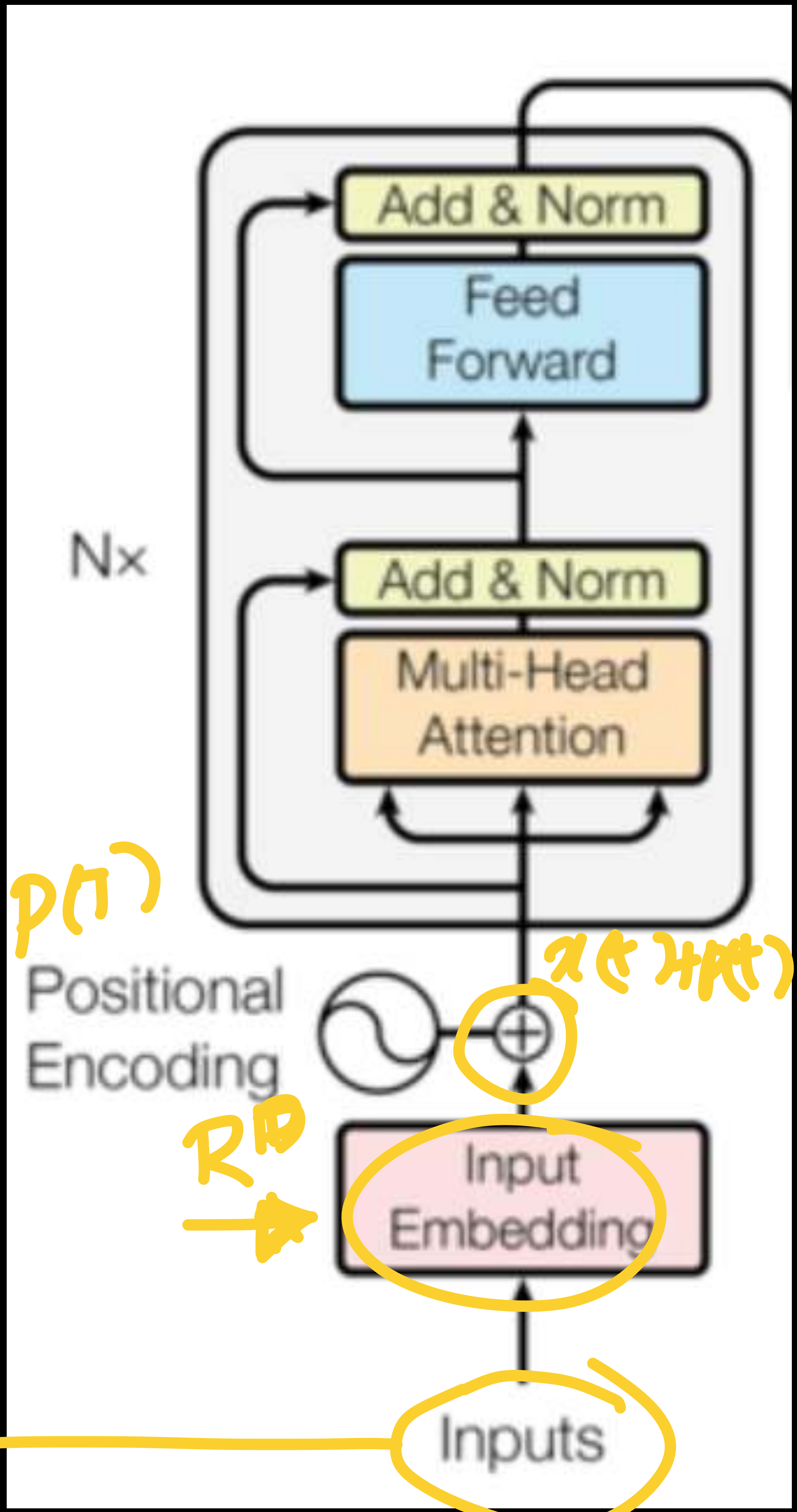
Dim →

(1), (2), (3) ... (T)

P(1) P(2) P(3)

Transformer encoder - overview

Reading Assignment - “Attention is All You Need”
https://arxiv.org/pdf/1706.03762.pdf



P(1)... P(T)

$x(t)+P(t)$

RD

T, D

$X = \{x(1) \dots x(T)\}$

128D

Transformer decoder

$[\hat{y}(s) \dots \hat{y}(1)]$

$[x(1) \dots x(T)]$

$[\hat{y}(1) \dots \hat{y}(s)]$

- Masked self-attention layer -
  - ✓ Mask makes the output dependecies causal
    - ★ Only the past is used to encode the attention.

$[\hat{y}(1) \dots \hat{y}(s+1)]$ $\hat{y}(s+2)$

	<s>	der	schnelle	braune	fuchs	</s>
<s>						
der						
schnelle			100	20	80	
braune				100	80	
fuchs						
</s>						

$Q \times K^T / \sqrt{d}$

\+

$e^{-inf} = 0$

0	-inf	-inf	-inf	-inf	-inf
0	0	-inf	-inf	-inf	-inf
0	0	0	-inf	-inf	-inf
0	0	0	0	-inf	-inf
0	0	0	0	0	-inf
0	0	0	0	0	0

Mask Shape = TxT = 6x6

=

	<s>	der	schnelle	braune	fuchs	</s>
<s>		-inf	-inf	-inf	-inf	-inf
der			inf	-inf	-inf	-inf
schnelle			100	-inf	-inf	-inf
braune				100	-inf	-inf
fuchs						-inf
</s>						

$Q \times K^T / \sqrt{d}$

Source - https://lionbridge.ai/articles/what-are-transformer-models-in-machine-learning/

Transformer decoder

- Masked self-attention layer -
    - ✓ Mask makes the output dependencies causal
        - ★ Only the past is used to encode the attention.

$$Softmax\left\{\frac{\mathbf{QK}^T}{\sqrt{d}}\right\}\mathbf{V} \xrightarrow{A_h^p} Softmax\left\{Mask + \frac{\mathbf{QK}^T}{\sqrt{d}}\right\}\mathbf{V}$$

- ★ Make the attention matrix to be lower triangular

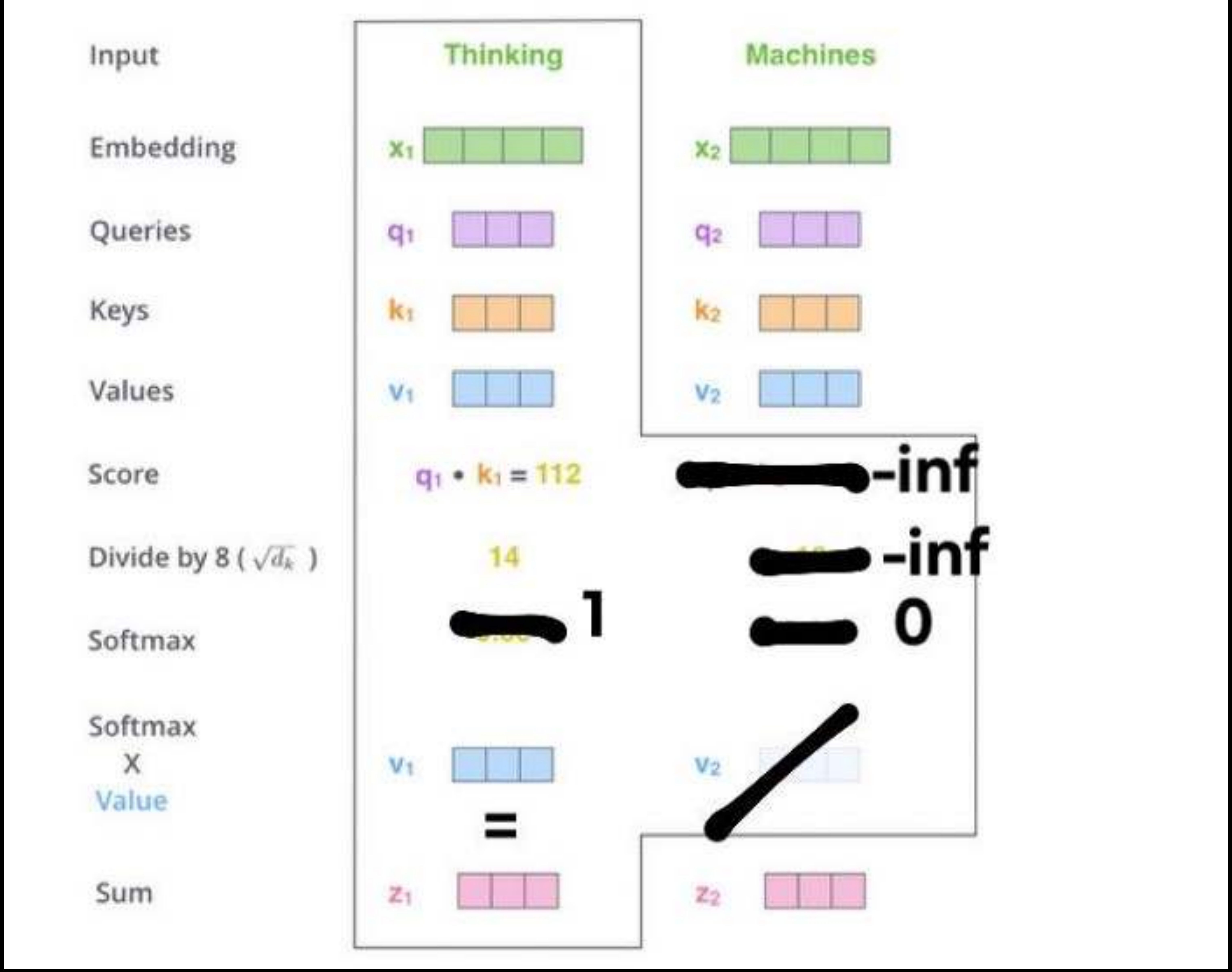
Transformer decoder

- Masked self-attention layer -
  - ✓ Mask makes the output dependecies causal
    - ★ Only the past is used to encode the attention.

	<s>	der	schnelle	braune	fuchs	</s>
<s>		0	0	0	0	0
der			0	0	0	0
schnelle			.75	0	0	0
braune				.85	0	0
fuchs						0
</s>						

softmax(QxK^T / $\sqrt{d}$)

Transformer decoder



Encoder-decoder attention

$X = \{x(1) \dots x(T)\}$ $T \times D$

$\hat{Y} = \{\hat{y}(1) \dots \hat{y}(S)\}$ $S \times D.$

- Use the Key and Value matrices from the last layer of the encoder

$$\mathbf{Q}_h^p = \mathbf{D}^{p-1}\mathbf{W}_h^{p,Q} + \mathbf{1}(\mathbf{b}_h^{p,Q})^T \in \mathcal{R}^{S\times d}$$

$$\mathbf{K}_h^p = \mathbf{E}^L\mathbf{W}_h^{p,K} + \mathbf{1}(\mathbf{b}_h^{p,K})^T \in \mathcal{R}^{T\times d}$$

$$\mathbf{V}_h^p = \mathbf{E}^L\mathbf{W}_h^{p,V} + \mathbf{1}(\mathbf{b}_h^{p,V})^T \in \mathcal{R}^{T\times d}$$

$p = 1 \dots P$ decoder layer

$$\mathbf{D}_h^p = softmax\left(\frac{\mathbf{Q}_h^p(\mathbf{K}_h^p)^T}{\sqrt{d}}\right)\mathbf{V}_h^p \in \mathcal{R}^{S\times d}$$

$S \times T$

$(S\times d)(d\times d) \rightarrow S\times d$

$S \times D$

$\boxed{h = \{1..H\}}$ heads $\boxed{d = \frac{D}{H}}$

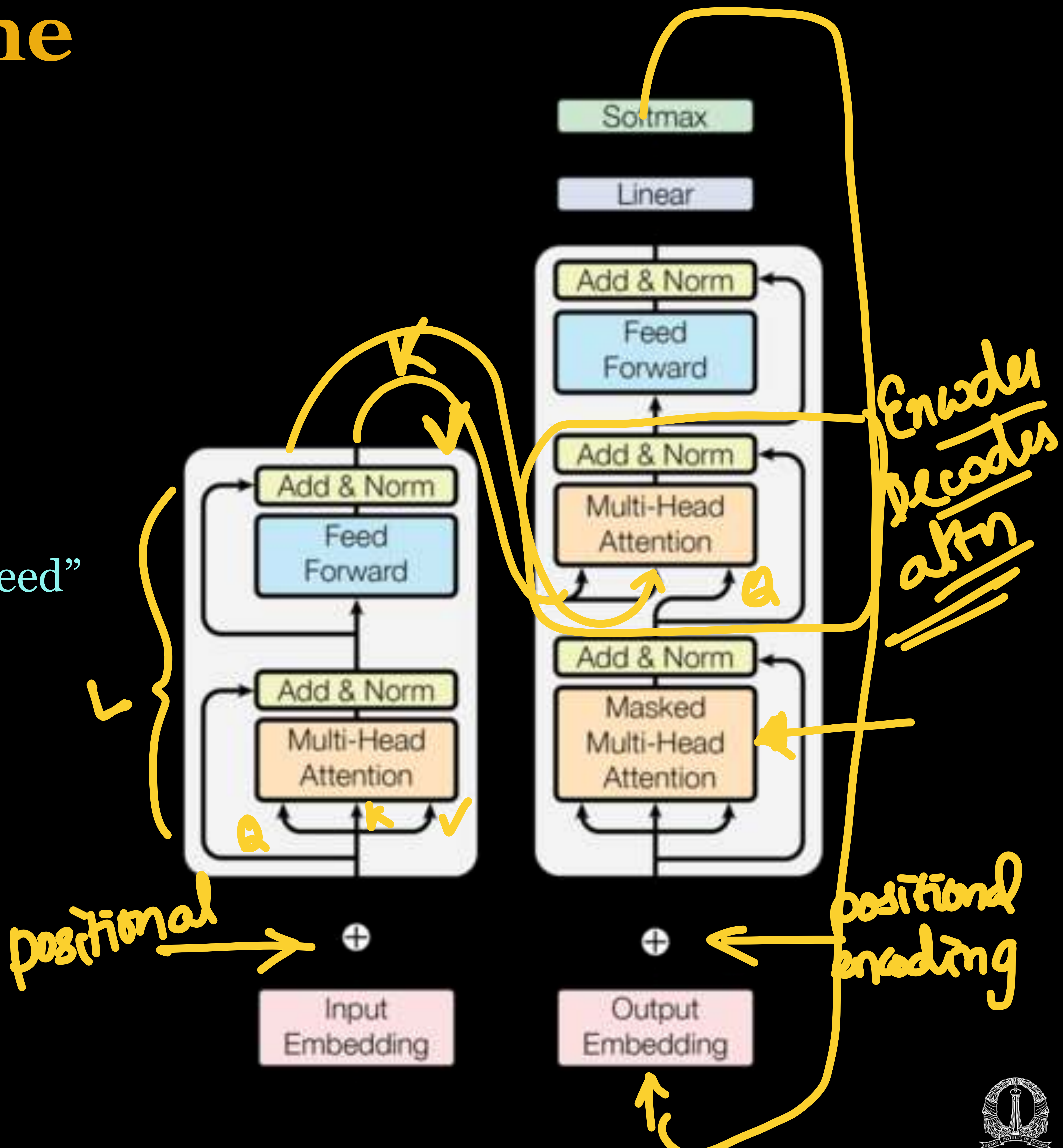

The quick brownfox
x(t)
brown
w
V
x(t-1)
w
softmax
quick {2000}
e(t)
w
x(t+1)
softmax V
One hot
embeddings fox {8862}
D
D << V
Eng V = 10K
V1
L1
L2
V2
V = { V1 + V2 }

Transformer - decoder

- Decoder Layer Output

- $$[\mathbf{d}^p(1)\ldots\mathbf{d}^p(S)] = ReLU\left(\mathbf{D}^p_{ff}\mathbf{W}^p_{of} + \mathbf{1}(\mathbf{b}^p_{of})^T\right) \in \mathcal{R}^{S\times D}$$

Transformer - full pipeline



Reading Assignment - “Attention is All You Need”
https://arxiv.org/pdf/1706.03762.pdf

Transformer - training

- Input sequence and the output sequence shifted right by 1

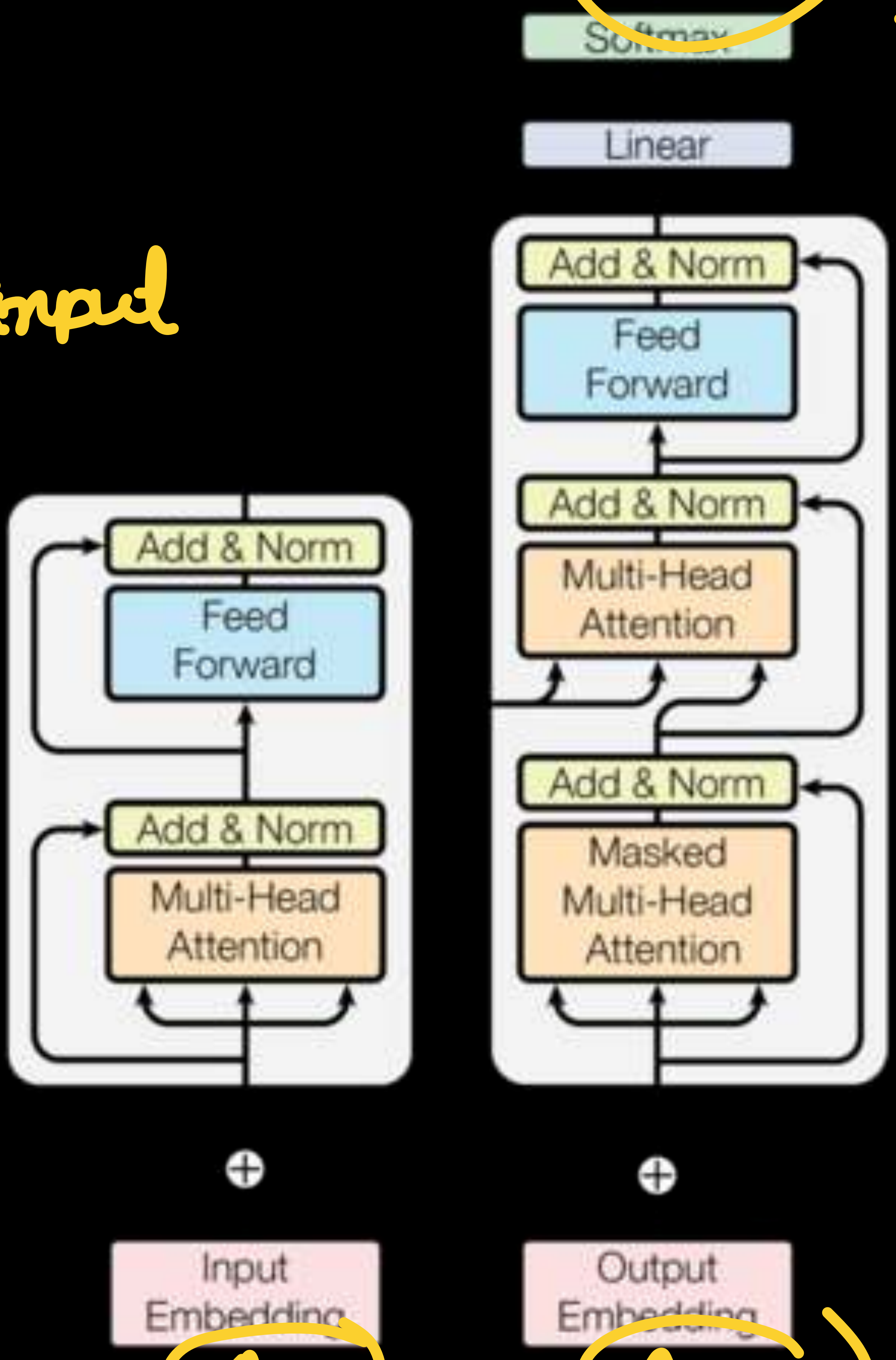
I The quick brown fox English input

II <s> der schnelle . . . German target

*

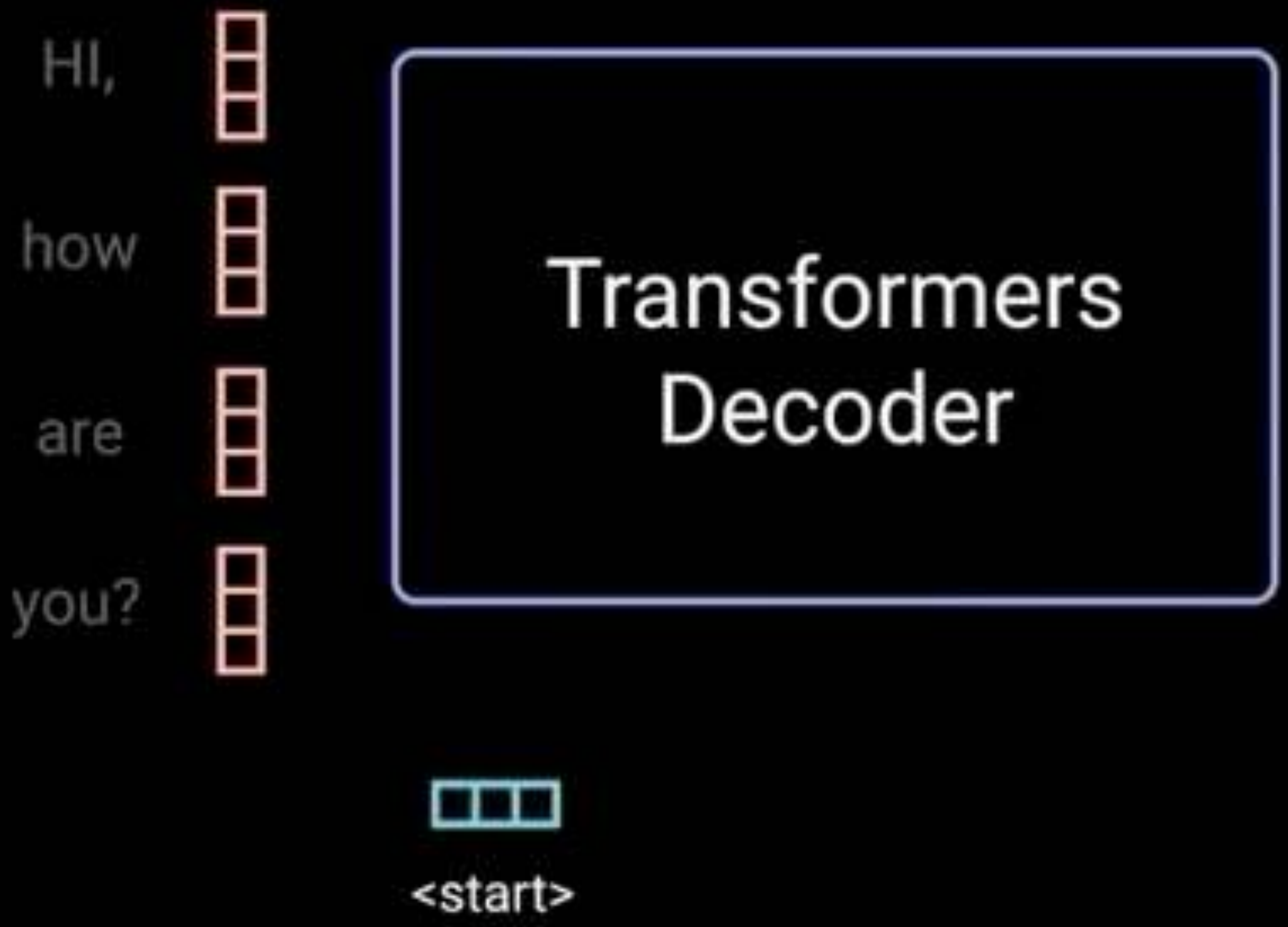
4

* *



Transformer decoder

- Inference



Greedy decoding

Source - https://lionbridge.ai/articles/what-are-transformer-models-in-machine-learning/

Topics thus far …

Visual and Time Series Modeling: Semantic Models, Recurrent neural models and LSTM models, Encoder-decoder models, Attention models.

Representation Learning, Causality And Explainability: t-SNE visualization, Hierarchical Representation, semantic embeddings, gradient and perturbation analysis, Topics in Explainable learning, Structural causal models.

Unsupervised Learning: Restricted Boltzmann Machines, Variational Autoencoders, Generative Adversarial Networks.

New Architectures: Capsule networks, End-to-end models, Transformer Networks.

Applications: Applications in NLP, Speech, Image/Video domains in all modules.